

PostgreSQL dans Docker : explorer et nettoyer la base DataShare

La base de DataShare est un PostgreSQL 16 qui tourne dans un conteneur : on n’a donc pas de client `psql` installé sur la machine, on exécute celui du conteneur avec `docker compose exec`. L’enjeu de la fiche : savoir **inspecter** la base, **lire** les données et **supprimer** des tuples (lignes) sans casser le schéma ni laisser le stockage des fichiers incohérent avec la base.

Se connecter

Le service s’appelle `db` dans `deploy/docker-compose.yml` (image `postgres:16-alpine`, base, utilisateur et mot de passe `datashare` par défaut). Les commandes se lancent depuis le dossier `deploy/`, là où se trouve le fichier `compose` :

```
docker compose exec db psql -U datashare -d datashare # shell interactif
docker compose exec db psql -U datashare -d datashare -c "SELECT now();" # requête ponctuelle
docker exec -it <conteneur> psql -U datashare -d datashare # sans compose (nom via docker ps)
```

`-c` exécute une seule requête SQL (Structured Query Language) puis rend la main ; sans `-c`, on entre dans le shell `psql`, qu’on quitte avec `\q`.

Méta-commandes `psql` et requêtes SQL

Deux familles à ne pas confondre dans le shell `psql` :

	Méta-commandes <code>psql</code>	Requêtes SQL
Syntaxe	commencent par <code>\</code> , sans <code>;</code>	se terminent par <code>;</code>
Rôle	explorer (bases, tables, structure)	lire ou modifier les données
Exemples	<code>\l</code> , <code>\dt</code> , <code>\d stored_file</code>	<code>SELECT ...;</code> , <code>DELETE ...;</code>
Qui les comprend	le client <code>psql</code> seulement	le serveur PostgreSQL

<code>\l</code>	lister les bases
<code>\dt</code>	lister les tables
<code>\d users</code>	structure d'une table (colonnes, index, clés)
<code>\di</code>	lister les index
<code>\x</code>	affichage vertical, pratique pour les lignes larges

Le schéma réel

Deux tables, créées par la migration Flyway `V1_init.sql` :

```

CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       VARCHAR(320) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE stored_file (
  ...
  storage_key  VARCHAR(255) NOT NULL,    -- clé de l'objet dans le stockage
  owner_id     UUID REFERENCES users(id) ON DELETE CASCADE, -- NULL = upload anonyme
  expires_at   TIMESTAMPTZ NOT NULL
);

```

- `UUID` (Universally Unique Identifier) : identifiant de 128 bits, généré par la base (`gen_random_uuid()`), non devinable contrairement à un entier auto-incrémenté.
- `stored_file.owner_id` est une clé étrangère vers `users(id)` avec `ON DELETE CASCADE` : supprimer un utilisateur supprime ses fichiers.
- `storage_key` désigne l'objet dans MinIO (stockage compatible S3, Simple Storage Service) : le fichier lui-même n'est **pas** dans PostgreSQL, seulement ses métadonnées.
- La table `flyway_schema_history` est tenue par Flyway : elle liste les migrations appliquées.

Lire les données

```

SELECT id, email, created_at FROM users;           -- sans le hash du mot de passe
SELECT original_name, size_bytes, expires_at FROM stored_file ORDER BY created_at DESC;
SELECT id, original_name FROM stored_file WHERE expires_at < now(); -- expirés
SELECT id, original_name FROM stored_file WHERE owner_id IS NULL;   -- uploads anonymes
SELECT version, description, success FROM flyway_schema_history;    -- migrations

```

Supprimer des tuples

Besoin	Commande	Effet
Une ligne précise	<code>DELETE FROM stored_file WHERE id = '<uuid>';</code>	supprime cette ligne
Les expirés	<code>DELETE FROM stored_file WHERE expires_at < now();</code>	supprime les lignes, pas les objets MinIO
Un utilisateur	<code>DELETE FROM users WHERE email = '...';</code>	supprime aussi ses fichiers (CASCADE)
Tout le contenu	<code>DELETE FROM stored_file;</code>	vide la table, elle reste
Vider vite	<code>TRUNCATE users CASCADE;</code>	vide <code>users</code> et <code>stored_file</code>

Un `DELETE` sans `WHERE` vide toute la table. Pour s'entraîner sans risque, on encadre par une transaction : rien n'est définitif tant qu'on n'a pas validé.

```
BEGIN;
DELETE FROM stored_file WHERE expires_at < now();
SELECT count(*) FROM stored_file;
ROLLBACK;  -- annule ; COMMIT; pour valider
```

Depuis Docker Desktop (onglet Exec)

Dans Docker Desktop, ouvrir **Containers**, cliquer sur le conteneur de la base (`datashare-dev-db-1`), puis l'onglet **Exec**. On est déjà **dans** le conteneur, dans un shell `sh` : on n'écrit donc ni `docker compose` ni `exec db`, seulement `psql`.

```
psql -U datashare -d datashare      # ouvre le shell psql (invite : datashare=#)
```

Puis, dans `psql` (méta-commandes sans `;`, SQL avec `;`) :

```
\dt                                -- lister les tables
SELECT * FROM users;                -- contenu de users
SELECT * FROM stored_file;          -- contenu de stored_file
SELECT id, original_name, expires_at FROM stored_file;  -- version lisible

-- supprimer un fichier à partir de son nom
SELECT id, original_name FROM stored_file WHERE original_name = 'rapport.pdf'; -- vérifier d'abord
DELETE FROM stored_file WHERE original_name = 'rapport.pdf';
SELECT id, original_name FROM stored_file;  -- contrôler le résultat

\q                                  -- quitter psql
```

Sans ouvrir le shell `psql`, une commande unique avec `-c` fonctionne aussi dans l'onglet :

```
psql -U datashare -d datashare -c "\dt"
psql -U datashare -d datashare -c "SELECT * FROM stored_file;"
psql -U datashare -d datashare -c "DELETE FROM stored_file WHERE original_name = 'rapport.pdf';"
```

Précautions propres à `original_name` : rien ne le rend unique, deux uploads peuvent porter le même nom, et le `DELETE` les supprime alors **tous les deux**. Le `SELECT` préalable montre combien de lignes sont concernées ; pour n'en supprimer qu'une, filtrer sur `id` (UUID) plutôt que sur le nom. Pour un nom partiel, `WHERE original_name ILIKE '%rapport%'` (insensible à la casse). Comme pour tout `DELETE` manuel, l'objet MinIO correspondant reste orphelin.

Sauvegarder et restaurer

```
docker compose exec -T db pg_dump -U datashare datashare > sauvegarde.sql
docker compose exec -T db psql -U datashare -d datashare < sauvegarde.sql
docker compose down -v      # efface aussi le volume db-data : reset total
```

-T désactive le pseudo-terminal, nécessaire pour rediriger l'entrée ou la sortie.

Dans DataShare

- Le service `db`, son volume `db-data` et son healthcheck `pg_isready` : [deploy/docker-compose.yml:9-21](#).
- Schéma et `ON DELETE CASCADE` : [V1_init.sql:11-21](#).
- `ddl-auto: validate` : Hibernate ne modifie jamais le schéma, il vérifie seulement qu'il correspond aux entités ; le DDL (Data Definition Language) vient exclusivement de Flyway ([application.yml:11-13](#)). Supprimer `flyway_schema_history` ou modifier une migration déjà appliquée fait échouer le démarrage.
- La suppression applicative, [FileService.delete](#), retire la ligne **puis** l'objet MinIO (un échec du stockage laisse un orphelin inoffensif). Un `DELETE` à la main ne fait que la première moitié.
- Pour purger les expirés correctement (ligne **et** objet MinIO), le projet fournit [deploy/purge-expired.sh](#), décrit dans [MAINTENANCE.md](#) (US10, déclenchement manuel en attendant `@Scheduled`).

À retenir

1. On n'installe pas `psql` : on utilise celui du conteneur, `docker compose exec db psql -U datashare -d datashare`.
2. `\dt`, `\d table` explorent, `SELECT` lit : les méta-commandes commencent par `\` et ne s'écrivent pas avec `;`.
3. Supprimer un `users` supprime ses fichiers (`ON DELETE CASCADE`) ; `TRUNCATE ... CASCADE` aussi.
4. La base ne contient que les métadonnées : un `DELETE` manuel laisse l'objet MinIO orphelin, d'où `purge-expired.sh` et `FileService.delete`.
5. On ne touche ni à `flyway_schema_history` ni aux migrations appliquées ; avant de supprimer : `SELECT` avec le même `WHERE`, ou `BEGIN / ROLLBACK`.